AUTOMATED SOFTWARE TESTING

Project Report

GROUP MEMBERS

NAWAL FATIMA BSE233211

NIMRA ABID BSE233075

SHIZA TARIQ BSE233175

SUBMITTED TO:

MA'AM SYEDA HAFSA

Table of Contents

Table of Figures:

1. System Description

1.1. Project Title

EventSys – Event Booking System

1.2. System Overview

EventSys is a web-based event booking system developed using PHP, MySQL, HTML, CSS, JavaScript, and Bootstrap. The system allows users to browse available events, search events, register accounts, log in securely, book tickets, manage bookings, and view purchased tickets.

Event organizers can create and manage events through the platform.

The application was selected because it contains multiple user workflows, business logic, form validations, database operations, and user interactions, making it suitable for automated software testing.

1.3.Key Features

- User Registration
- User Login and Logout
- Event Search
- Event Browsing
- Ticket Booking
- Checkout Processing
- Booking Confirmation
- My Tickets Management
- Event Creation and Management

1.4. Assumptions

- Users must be registered before booking tickets.

- Users can only access their own bookings.
- Ticket quantity must remain within the allowed range.
- Required fields must be completed before checkout.
- Successful booking generates a valid booking record.

2. Requirement and System Understanding

2.1. Core Features Selected for Testing

The following critical features were selected because they directly affect system correctness and user experience:

1. User Authentication
2. Event Search
3. Ticket Quantity Validation
4. Price Calculation
5. Event Validation
6. Ticket Booking Workflow
7. Checkout Processing
8. Booking Confirmation

2.2. Input–Output Relationships

Input	Processing	Output
Login Credentials	Verify user information	Login Success/Failure
Search Keyword	Search event records	Matching Events
Ticket Quantity	Validate quantity	Accept/Reject
Event Data	Validate event information	Event Created/Rejected

Booking Request	Process booking	Booking Confirmation

2.3. System Constraints

- Ticket quantity must be between 1 and 4.
- Login credentials must be valid.
- Event information cannot contain empty required fields.
- Users must be logged in before booking.
- Phone number is mandatory during checkout.

2.4. System interface screenshots:

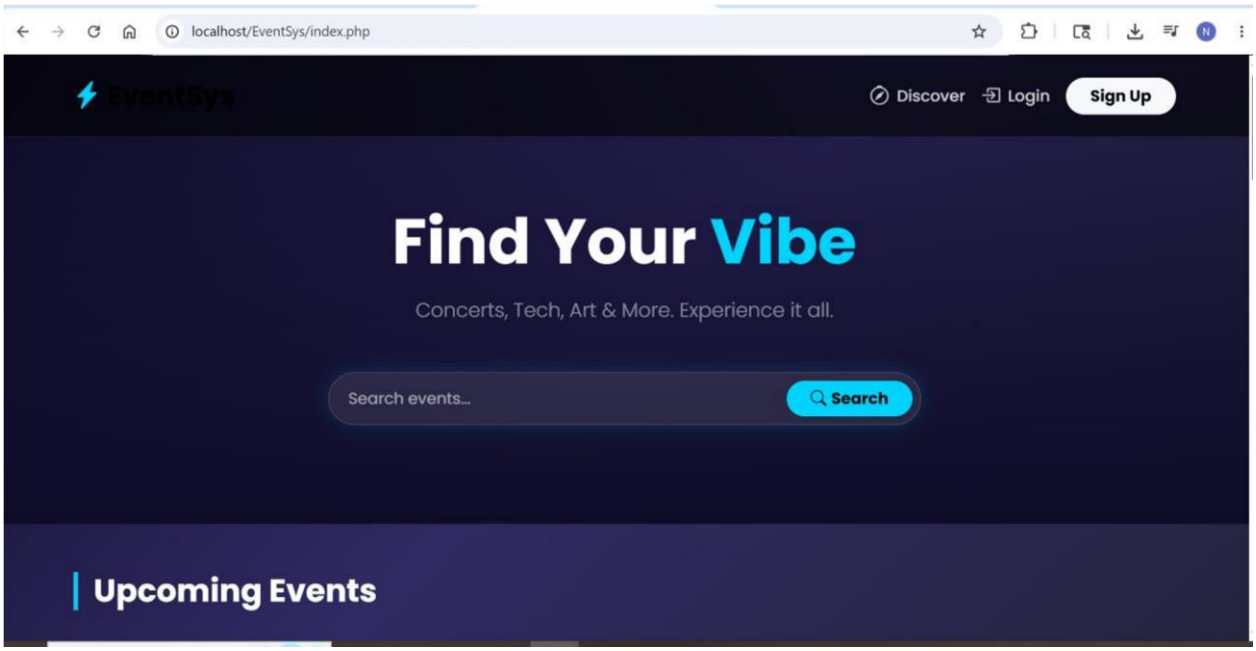


Figure 1: Homepage / Event Listing (index.php)

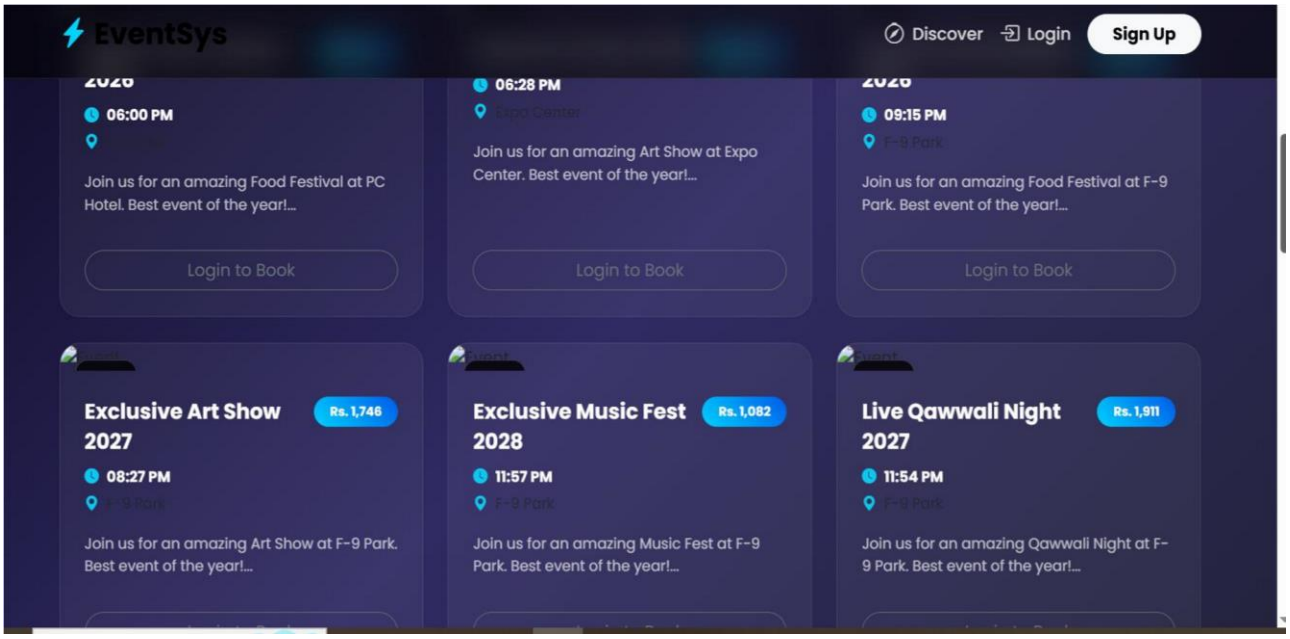


Figure 2: Upcommming Events

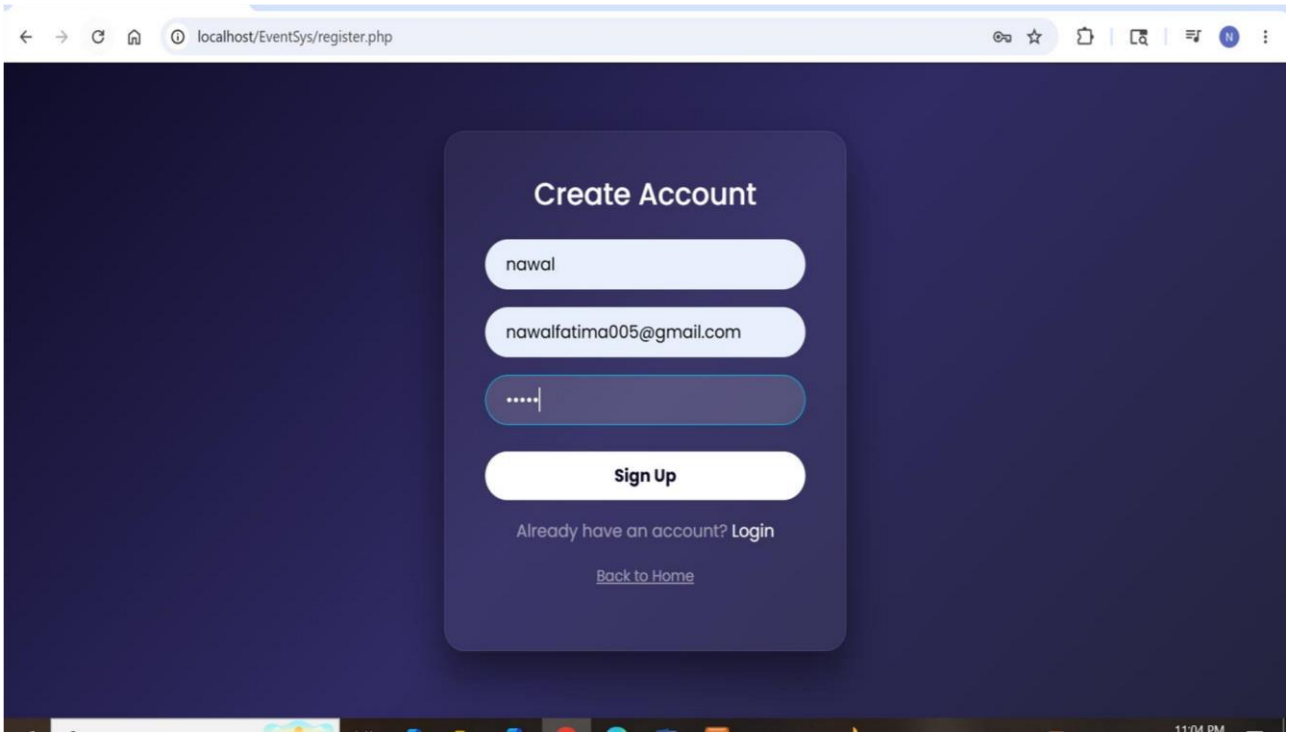


Figure 3: User Registration (register.php)

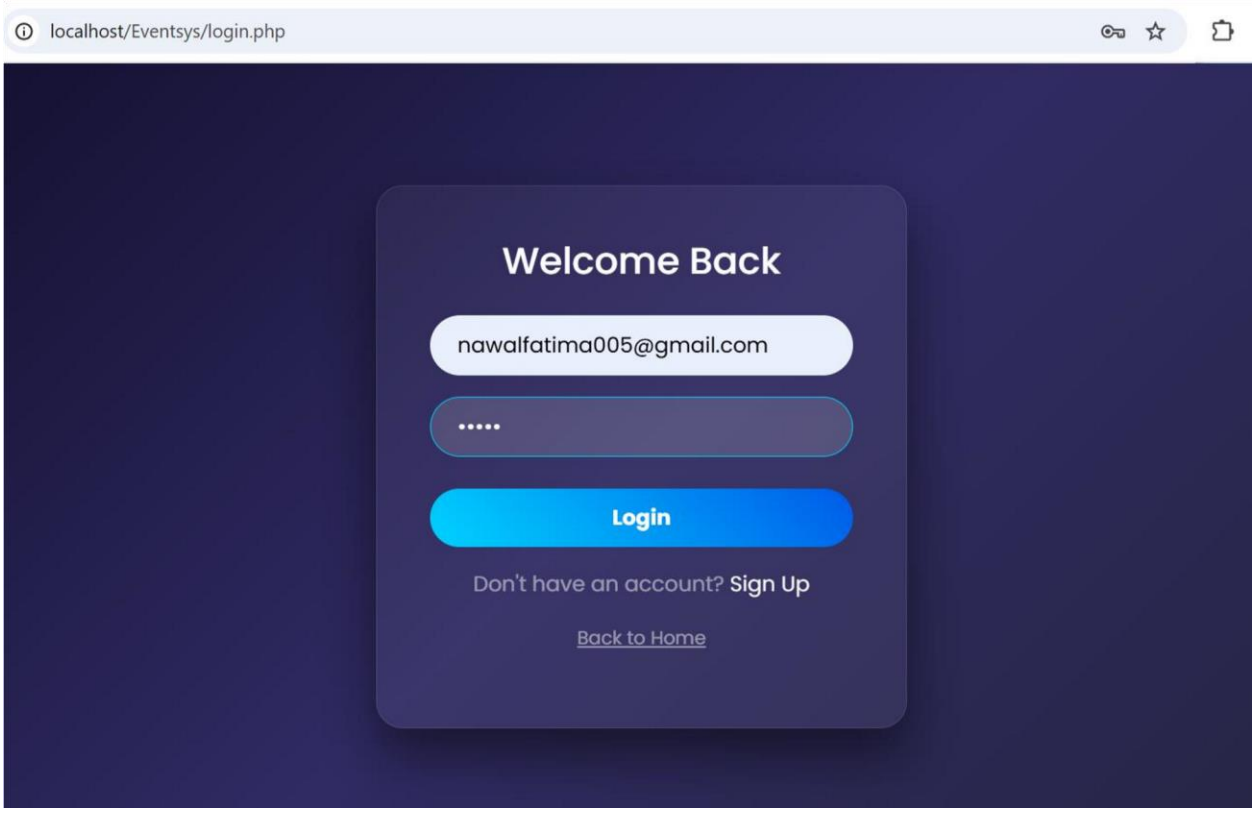


Figure 4: User Login (login.php)

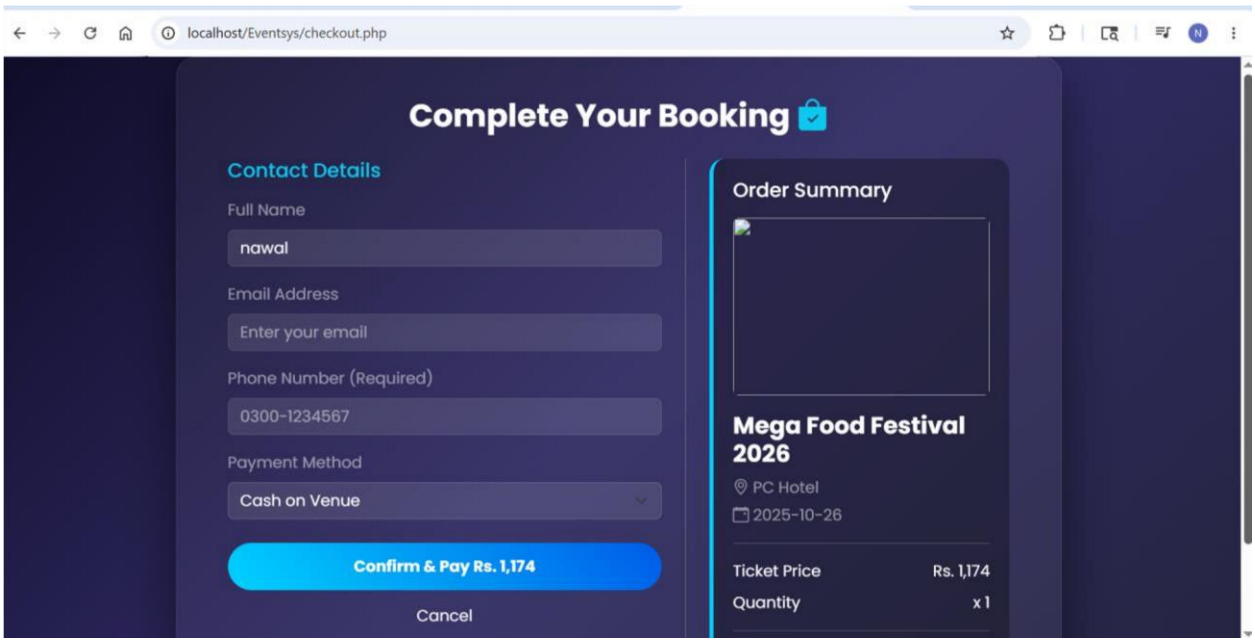


Figure 5: Checkout Page (checkout.php)

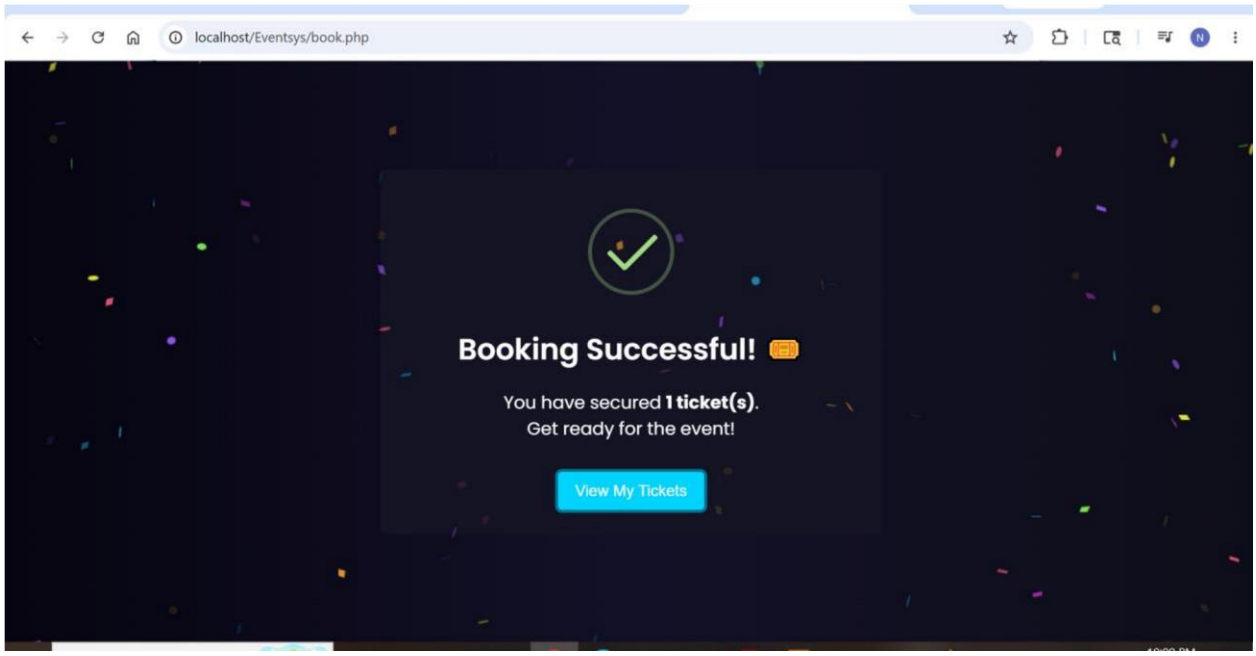


Figure 6: My Bookings (my_bookings.php)

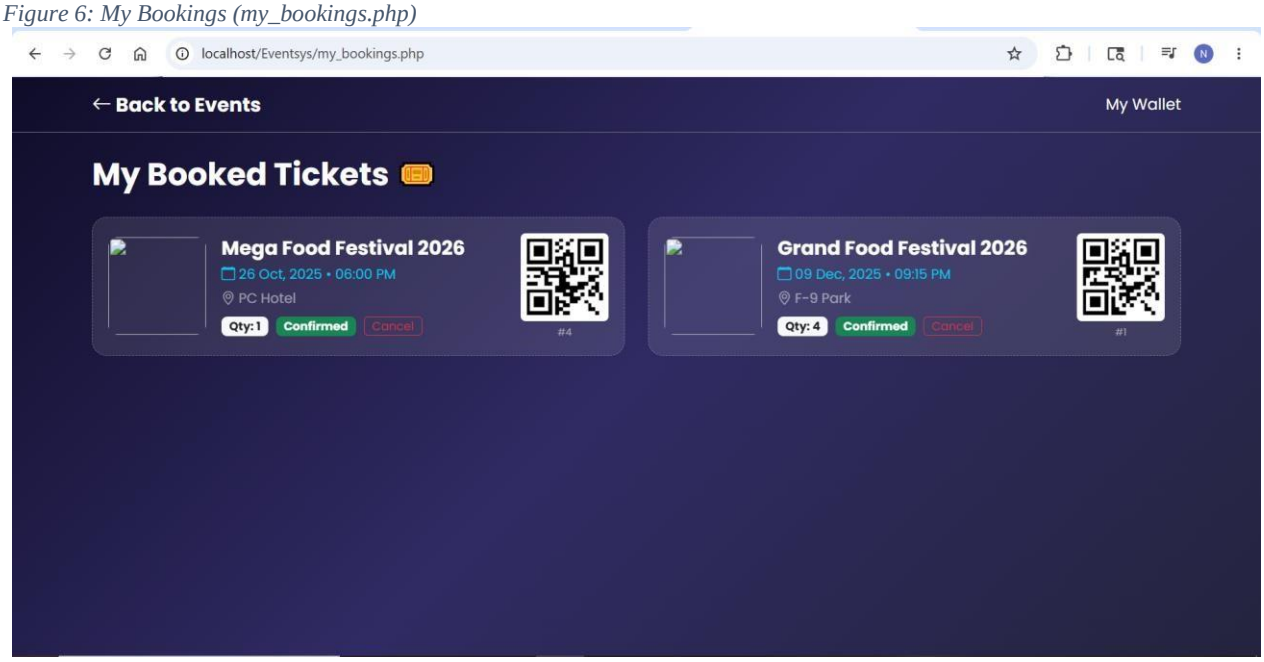


Figure 7: Booked Tickets

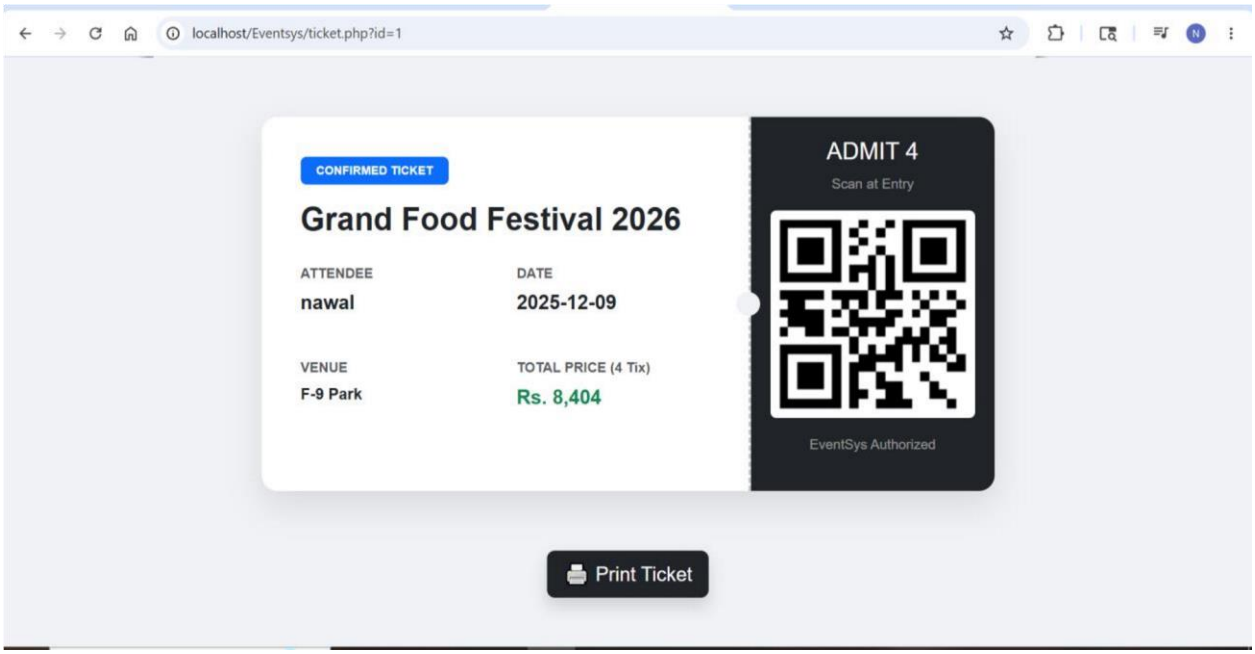


Figure 8: Printable Ticket (ticket.php)

3. Test Strategy

3.1. Test Levels Applied

3.1.1. Unit Testing

Used to verify individual business logic functions separately.

3.1.2. UI Testing

Used to verify user interface behavior and interactions.

3.1.3. End-to-End Testing

Used to validate the complete booking workflow from login to booking confirmation.

3.2. Test Types Covered

3.2.1. Functional Testing

Verifies that system functions work according to requirements.

3.2.2. Boundary Testing

Checks minimum and maximum valid input values.

Example:

- Quantity = 1
- Quantity = 4

3.2.3. Negative Testing

Checks invalid and unexpected inputs.

Example:

- Quantity = 0
- Quantity = 5
- Invalid Login Credentials

3.2.4. State-Based Testing

Verifies behavior after state changes.

Example:

- Login Success → Dashboard Accessible
- Booking Success → Confirmation Displayed

3.2.5 Property-Based Testing

Property-based testing was performed to verify that important system properties remain valid across multiple inputs. The price calculation functionality was tested to ensure that the total booking amount always equals the ticket price multiplied by the selected ticket quantity.

4. Test Case Design

The test cases were designed to verify normal conditions, boundary values, invalid inputs, and workflow transitions. Each test case was created with a clearly defined expected result to ensure objective verification.

The testing process covered both individual system components and complete user workflows.

Test Level	Number of Tests
Unit Testing	15
UI Testing	3
End-to-End Testing	1
Total	19

5. Unit Testing

5.1 Unit Testing Overview

Unit testing was performed using PHPUnit to verify critical business logic functions of the EventSys application. The purpose of unit testing was to ensure that individual functions behave correctly before integration with other system components.

The implementation and execution of these tests were completed as part of Assignment 3, which served as the first phase of this Automated Software Testing project.

5.2 Components Tested

Component	Purpose
Ticket Quantity Validation	Verify valid ticket quantity limits
Price Calculation	Verify total booking amount
Event Validation	Verify event information correctness
Event Search	Verify search functionality
Password Verification	Verify authentication logic

5.3 Unit Test Cases Summary

Test ID	Scenario	Expected Result
UT-01	Quantity = 1	Valid
UT-02	Quantity = 4	Valid
UT-03	Quantity = 0	Invalid
UT-04	Quantity = 5	Invalid
UT-05	Valid Price Calculation	Correct Total
UT-06	Empty Event Title	Invalid
UT-07	Valid Event Data	Accepted
UT-08	Existing Search Keyword	Event Returned
UT-09	Correct Password	Verified
UT-10	Incorrect Password	Rejected

5.4 Oracle Design for Unit Testing

The correctness of each unit test was verified using expected outputs and business rules.

Test Case	What Is Checked	Why It Is Correct
Ticket Quantity Validation	Quantity between 1 and 4	Business rule defines valid range
Price Calculation	Total = Price × Quantity	Mathematical formula defines correct result
Event Validation	Required fields present	Event cannot be created without valid data
Event Search	Matching events returned	Search functionality should return relevant results
Password Verification	Password hash matches	Confirms authentication correctness

5.4.1 Property-Based Testing

Objective

Property-Based Testing was performed to validate the correctness of the ticket price calculation logic across multiple valid inputs.

Property Tested

For every valid ticket quantity between 1 and 4:

Total Price = Ticket Price × Quantity

Implementation

A PHPUnit test was implemented to automatically evaluate the price calculation function using all valid ticket quantities within the allowed range. The test iterated through quantities from 1 to 4 and verified that the calculated total price matched the expected mathematical result for each input.

The property-based test was implemented in the EventFunctionsTest.php file using PHPUnit assertions.

```
    public function testPriceCalculationProperty()
    {
    $price = 1500;

    for ($quantity = 1; $quantity <= 4; $quantity++) {

        $expected = $price * $quantity;

        $this->assertEquals(
            $expected,
            EventFunctions::calculateTotalPrice(
                $price,
                $quantity
            )
        );
    }
    }
}
```

Figure 9 (property based test implementation)

Oracle Design for Property-Based Testing

Property Checked	Correctness Criteria	Reason for Correctness
Total Price Calculation	Calculated Total = Ticket Price × Quantity	The booking system calculates cost based on the number of tickets selected. Therefore, the total amount must always equal the ticket price multiplied by the ticket quantity.

The oracle for this test was based on the mathematical relationship between ticket price and quantity. A test execution was considered successful only when the calculated result matched the expected value for every valid quantity.

Test Execution Result

The property-based test was executed successfully as part of the PHPUnit test suite. All assertions passed without failures, confirming that the price calculation logic satisfies the defined property for all tested inputs.

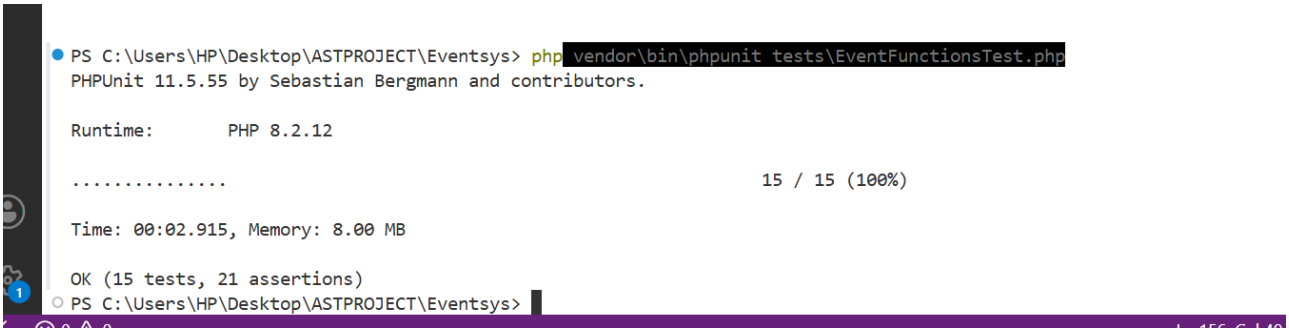


Figure 10 (Property-Based Test Execution Result)

Result Summary

The property was successfully verified for all valid ticket quantities. The successful execution of the test increased confidence in the correctness and reliability of the price calculation functionality by ensuring that the defined business rule remained valid across multiple inputs.

5.5 Automation Implementation

PHPUnit was used as the automation framework for unit testing. Test methods were implemented within EventFunctionsTest.php and executed using PHPUnit. Positive and negative test scenarios were included to validate business logic under different conditions.

Unit Test File – EventFunctionsTest.php

Code:

```
<?php

use PHPUnit\Framework\TestCase;

require_once __DIR__ . '/../src/EventFunctions.php';

class EventFunctionsTest extends TestCase
{     public function
testValidTicketQuantity()
   {
      $this->assertTrue(EventFunctions::isValidTicketQuantity(1));        $this-
>assertTrue(EventFunctions::isValidTicketQuantity(4));
   }

   public function testInvalidTicketQuantityLessThanOne()
```

```
{
    $this->assertFalse(EventFunctions::isValidTicketQuantity(0));
}

public function testInvalidTicketQuantityGreaterThanFour()
{
    $this->assertFalse(EventFunctions::isValidTicketQuantity(5));
}

public function testCalculateTotalPrice()
{
    $this->assertEquals(3000, EventFunctions::calculateTotalPrice(1500, 2));
}

public function testCalculateTotalPriceWithInvalidQuantity()
{
    $this->expectException(InvalidArgumentException::class);
    EventFunctions::calculateTotalPrice(1500, 5);
}

public function testCalculateTotalPriceWithNegativePrice()
{
    $this->expectException(InvalidArgumentException::class);
```

```
    EventFunctions::calculateTotalPrice(-1000, 2);
}

public function testValidEventData()
{
    $result = EventFunctions::isValidEventData(
        "Tech Expo",
        "Expo Center",
        1500,
        "2026-06-15",
        "18:00"
    );

    $this->assertTrue($result);
}

public function testInvalidEventDataWithEmptyTitle()
{
    $result = EventFunctions::isValidEventData(
        "",
        "Expo Center",
        1500,
        "2026-06-15",
```

```
        "18:00"
    );

    $this->assertFalse($result);
}

public function testInvalidEventDataWithZeroPrice()
{
    $result = EventFunctions::isValidEventData(
        "Tech Expo",
        "Expo Center",
        0,
        "2026-06-15",
        "18:00"
    );

    $this->assertFalse($result);
}

public function testSearchEventsByTitle()
{
    $events = [
        [
```

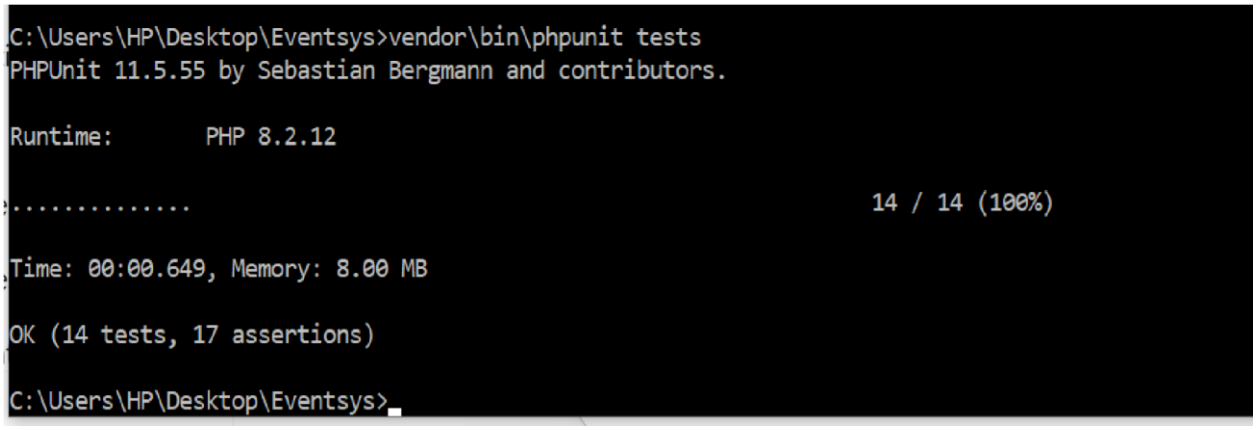
```
            "title" => "Tech Expo",
            "description" => "Technology event"
        ],
        [
            "title" => "Food Festival",
            "description" => "Food and family event"
        ]
    ];

    $result = EventFunctions::searchEvents($events, "tech");

    $this->assertCount(1, $result);
    $this->assertEquals("Tech Expo", $result[0]["title"]);
}

public function testSearchEventsByDescription()
{
    $events = [
        [
            "title" => "Qawwali Night",
            "description" => "Music event"
        ],
        [
```

```
            "title" => "AI Seminar",
            "description" => "Technology and innovation event"            ]
    ];

    $result = EventFunctions::searchEvents($events, "innovation");

    $this->assertCount(1, $result);
    $this->assertEquals("AI Seminar", $result[0]["title"]);
}

public function testSearchEventsNoResult()
{
    $events = [
        [
            "title" => "Qawwali Night",
            "description" => "Music event"
        ]
    ];

    $result = EventFunctions::searchEvents($events, "cricket");

    $this->assertCount(0, $result);
}
```

```
    public function testCorrectPasswordVerification()
    {
        $hashedPassword = password_hash("abc123", PASSWORD_DEFAULT);

        $this->assertTrue(EventFunctions::verifyPassword("abc123", $hashedPassword));
    }

    public function testIncorrectPasswordVerification()
    {
        $hashedPassword = password_hash("abc123", PASSWORD_DEFAULT);

        $this->assertFalse(EventFunctions::verifyPassword("wrongpass", $hashedPassword));
    }
}
```

5.6 Unit Testing Results

A total of fourteen unit test cases were executed. The test suite covered ticket quantity validation, price calculation, event validation, password verification, and event search functionality.



5.7 Unit Testing Summary

The unit testing phase successfully validated the core business logic of the EventSys application. All implemented PHPUnit test cases passed successfully and produced the expected results. The successful execution of these tests increased confidence in the correctness and reliability of the application's internal logic before proceeding to UI and End-to-End testing.

6. UI testing:

6.1 Tool Used

Tool: Selenium WebDriver (Python)

Purpose: To automate user interface testing of the EventSys web application.

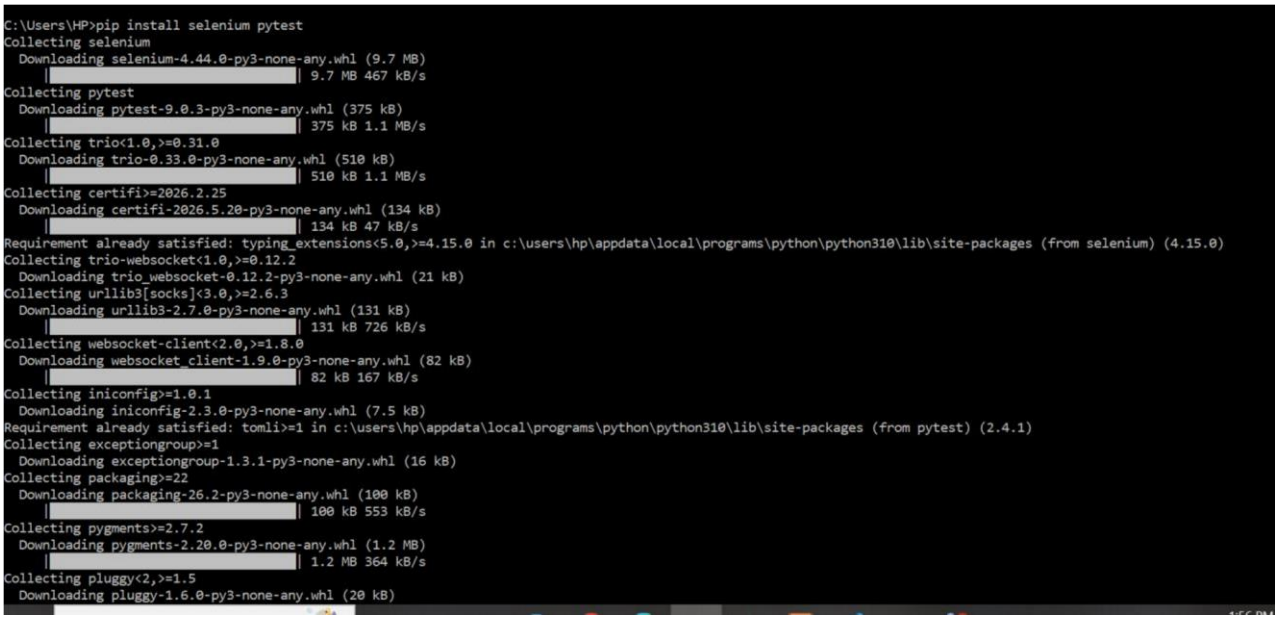


Figure 9: Test Environment Prepration and Downloading Dependencies

6.2. UI Test Cases Table

Test Case ID	Test Name	Steps	Expected Result	Actual Result
UI-01	Homepage Load	Open homepage	Homepage loads successfully	Pass
UI-02	Event Search	Search for an event	Matching events displayed	Pass
UI-03	User Login	Enter valid credentials and login	User logged in successfully	Pass

6.3. Selenium Code Screenshot

- **test_homepage_opens():**

```python
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

def test_homepage_opens():
    driver = webdriver.Chrome()
    driver.get("http://localhost/Eventsys/index.php")
    driver.maximize_window()

    time.sleep(2)

    assert "EventSys" in driver.page_source

    driver.quit()


```

- **test_search_event()**

```python

def test_search_event():
    driver = webdriver.Chrome()
    driver.get("http://localhost/Eventsys/index.php")
    driver.maximize_window()

    time.sleep(2)

    search_box = driver.find_element(By.NAME, "search")
    search_box.send_keys("food")
    search_box.submit()

    time.sleep(2)

    assert "Food" in driver.page_source or "food" in driver.page_source

    driver.quit()


```

- **test_user_login()**

```python
def test_user_login():
    driver = webdriver.Chrome()
    driver.get("http://localhost/Eventsys/login.php")
    driver.maximize_window()

    time.sleep(2)

    driver.find_element(By.NAME, "email").send_keys("nawalfatima005@gmail.com")
    driver.find_element(By.NAME, "password").send_keys("12345")
    driver.find_element(By.TAG_NAME, "button").click()

    time.sleep(2)

    assert "Logout" in driver.page_source or "index.php" in driver.current_url

    driver.quit()
```

Figure 10: Selenium UI Test Automation Code

6.4. Test Execution Screenshot

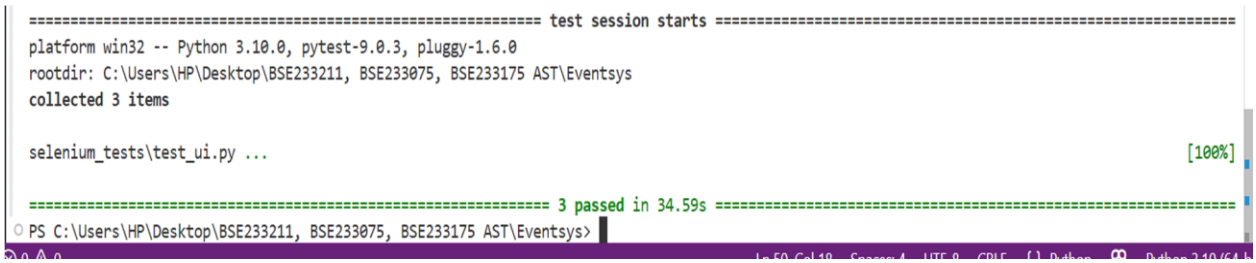


Figure 11: Selenium UI Test Execution Results

6.5. Oracle Design for UI Testing

Oracle Design defines the criteria used to determine whether a test has passed or failed. For each automated UI test, correctness is verified using observable outputs from the user interface. The selected oracles ensure that the system behaves according to the expected requirements and that each user interaction produces the correct outcome.

Test Case ID	Test Case	Oracle (Correctness Criteria)	Reason for Correctness

UI-01	Homepage Load Test	The homepage loads successfully and the text "EventSys" is present in the page source.	The presence of the application name confirms that the homepage has loaded correctly and the user interface is accessible.
UI-02	Event Search Test	After entering a search keyword, matching event information is displayed in the search results.	Displaying relevant search results confirms that the search functionality processes user input correctly and retrieves the expected events.
UI-03	User Login Test	The user successfully logs in and the application displays authenticated user access (successful navigation after login).	Successful authentication confirms that the login mechanism correctly validates user credentials and grants authorized access.

The UI test oracles were implemented using Selenium assertions. Each assertion verifies a specific expected outcome after a user action is performed. A test is considered successful only when the observed result matches the predefined correctness criteria. These oracles provide objective evidence that the tested functionality behaves according to the system requirements.

6.6. UI Testing Summary

Three UI test cases were automated using Selenium WebDriver. The tests verified homepage accessibility, event search functionality, and user login functionality. All automated UI tests executed successfully and passed without failures.

7. End-to-End Testing

7.1 Objective

End-to-End (E2E) testing was performed to validate the complete booking workflow of the EventSys application. The purpose of this testing was to ensure that all integrated components work together correctly from user login to successful ticket booking.

7.2 Tool Used

Tool	Purpose
Selenium WebDriver (Python)	Automate and validate the complete user booking workflow

7.3 End-to-End Test Scenario

Test Case ID	Test Scenario	Expected Result	Actual Result
E2E-01	Login → Select Event → Book Ticket → Checkout → Booking Successful	User successfully books a ticket and receives booking confirmation	Pass

7.4 Test Steps

1. Open the EventSys login page.
2. Enter valid user credentials.
3. Log in to the system.
4. Select an available event.
5. Click the **Book Ticket** button.
6. Navigate to the checkout page.
7. Enter the required phone number.
8. Click **Confirm & Pay**.
9. Verify that the **Booking Successful** message is displayed.

7.5 Oracle Design for End-to-End Testing

Oracle Design defines how correctness is verified during the execution of the complete booking workflow.

Test Case	Oracle (Correctness Criteria)	Reason for Correctness
E2E-01 Booking Workflow	The system displays the "Booking Successful" confirmation message after checkout.	The confirmation message indicates that login, event selection, checkout processing, and ticket booking were completed successfully.

The End-to-End test oracle was implemented using Selenium assertions. The test was considered successful only when the booking confirmation page was displayed after completing the entire booking process. This provides objective evidence that all major system components interact correctly and support the complete user workflow.

7.6 Automation Code

```
def test_end_to_end_booking():
    driver = webdriver.Chrome()
    driver.get("http://localhost/Eventsys/login.php")
    driver.maximize_window()
    time.sleep(2)
    driver.find_element(By.NAME, "email").send_keys("nawalfatima005@gmail.com")
    driver.find_element(By.NAME, "password").send_keys("12345")
    driver.find_element(By.TAG_NAME, "button").click()
    time.sleep(3)
    book_button = driver.find_element(By.XPATH, "//button[contains(., 'Book Ticket')]")
    driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", book_button)
    time.sleep(1)
    driver.execute_script("arguments[0].click();", book_button)
    time.sleep(3)
    driver.find_element(By.NAME, "phone").send_keys("03188958989")
    confirm_button = driver.find_element(By.XPATH, "//button[contains(., 'Confirm & Pay')]")
    driver.execute_script("arguments[0].scrollIntoView({block: 'center'});", confirm_button)
    time.sleep(1)
    driver.execute_script("arguments[0].click();", confirm_button)
    time.sleep(5)

    assert "Booking Successful" in driver.page_source

    driver.quit()
```

Figure 12: Selenium End-to-End Test Automation Code

7.7 Test Execution Result

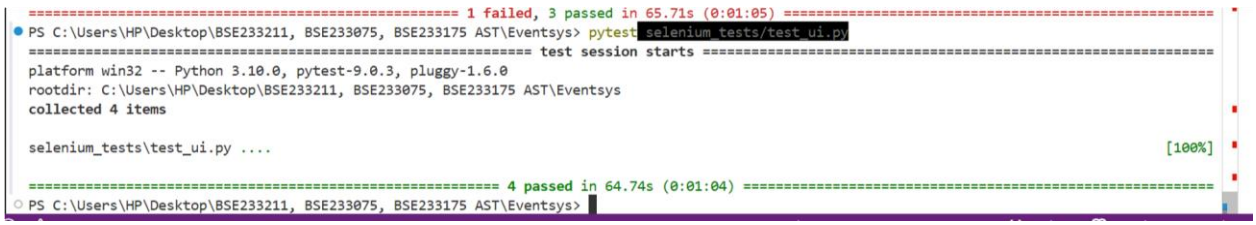


Figure 13: End-to-End Test Execution Result

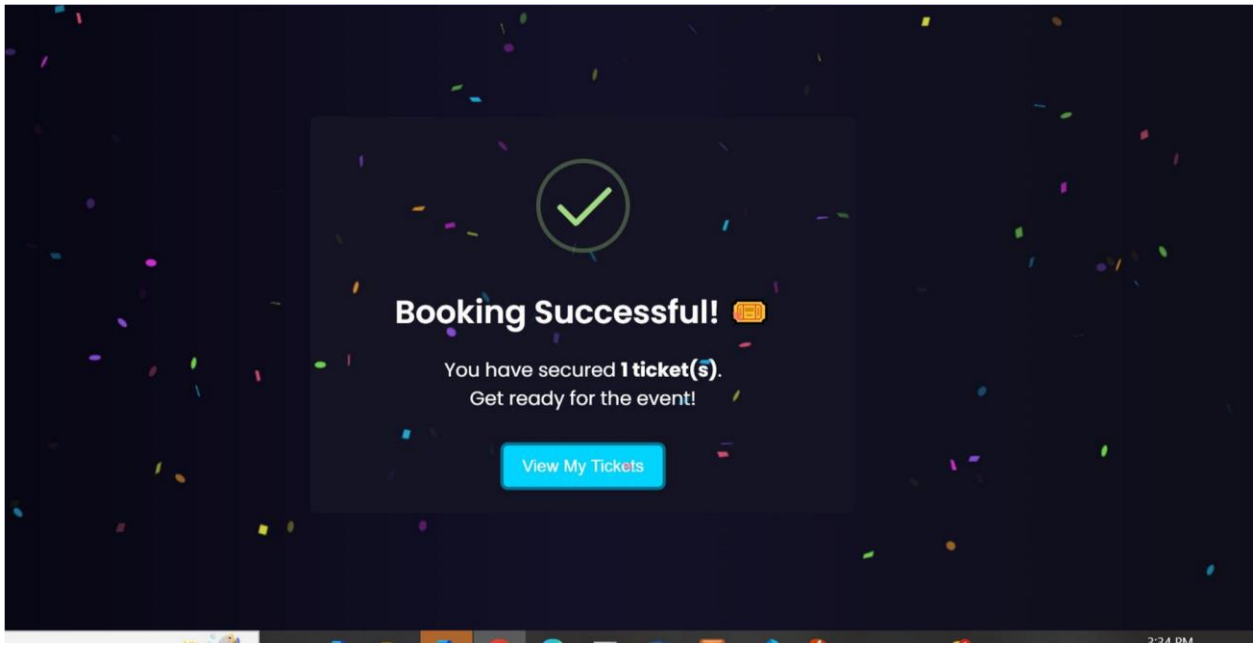


Figure 14: Successful Booking

7.8 End-to-End Testing Summary

One End-to-End test case was automated using Selenium WebDriver to validate the complete event booking workflow. The test covered user authentication, event selection, ticket booking, checkout processing, and booking confirmation. The automated test executed successfully and

passed without failures, demonstrating that the integrated components of the EventSys application work together correctly to support the ticket booking process.

8. Handling Flaky Tests

During the automation phase, some Selenium tests produced inconsistent results because the application pages were not fully loaded when Selenium attempted to interact with page elements. This issue was observed while performing the booking workflow, where buttons were sometimes not clickable immediately after page navigation.

Another issue occurred during checkout because the phone number field was mandatory. Initially, the automated script attempted to submit the form without entering a phone number, causing the booking process to stop at the checkout page.

To resolve these issues, appropriate waiting periods were added using time.sleep() to allow pages to load completely before interactions were performed. In addition, scrolling and JavaScriptbased clicks were used when required, and the checkout form was updated to automatically provide the required phone number before submission. After applying these improvements, all automated tests executed successfully and produced consistent results across multiple runs.

9. Test Adequacy Evaluation

The developed test suite provides sufficient coverage of the core functionalities of the EventSys application. Testing was performed at different levels to validate both individual components and complete system workflows.

Unit testing was used to verify critical business logic, including ticket quantity validation, total price calculation, event data validation, password verification, and event search functionality. UI testing was performed to ensure that important user interface features such as homepage accessibility, event search, and user login behaved correctly. End-to-End testing validated the complete booking process from user authentication to successful booking confirmation.

9.1. Functional Coverage

- User Registration
- User Login
- Event Search
- Ticket Quantity Validation
- Price Calculation
- Event Creation Validation
- Checkout Process
- Ticket Booking
- Booking Confirmation

9.2. Logical Coverage

- Positive Test Scenarios
- Negative Test Scenarios
- Boundary Value Scenarios
- User Workflow Validation

9.3. Limitations

Although the test suite covers the most important system functions, certain areas were not included within the project scope. These include performance testing, security testing, crossbrowser testing, and testing with multiple concurrent users. The application currently uses a simple booking workflow and does not integrate a real payment gateway; therefore, payment processing was not tested.

The implemented tests provide a strong level of confidence in the correctness of the EventSys application. However, as with any software testing activity, complete defect detection cannot be guaranteed.

10. Challenges and Reflection

One of the main challenges encountered during this project was that the original PHP application combined presentation logic, database operations, and business logic within the same files. This made direct unit testing difficult because individual functions could not be tested independently. To overcome this issue, key business logic was separated into a dedicated helper file, which allowed PHPUnit tests to be implemented more effectively.

Another challenge was encountered during Selenium automation. Certain page elements were not immediately available for interaction after page loading, which caused test failures during early execution attempts. Additional issues occurred during checkout because mandatory form fields were not initially handled by the automation script. These problems were resolved through careful debugging, the use of waiting mechanisms, and improvements to the automated test scripts.

This project provided practical experience with automated software testing techniques. It demonstrated the importance of unit testing for validating business logic, UI testing for verifying user interactions, and End-to-End testing for ensuring that complete workflows operate correctly. The project also highlighted the value of well-designed test cases and test oracles in improving software quality and reliability.

11. Conclusion

This project successfully applied automated software testing techniques to EventSys, a webbased event booking system. Testing activities included unit testing using PHPUnit, UI testing using Selenium WebDriver, and End-to-End testing of the complete booking workflow.

Critical functionalities such as user authentication, event search, ticket validation, price calculation, checkout processing, and booking confirmation were tested through automated test cases. The execution results showed that all implemented tests passed successfully and produced the expected outcomes.

The project demonstrated how automated testing can improve confidence in software quality by verifying both individual components and complete user workflows. The testing process also helped identify and resolve issues related to form validation and user interaction, resulting in a more reliable and stable application.